

NAME - NAKUL
ARORA

ROLL
NO - 2401010025

ASSIGNMENT 3

DBMS

SUBMITTED TO -

Mrs.
Mansi
Kajal.

ques 1. what is SQL? why is it important in DBMS?
Ans SQL stands for Structured Query Language. It is a standard language used to communicate with relational database management system. SQL allows user to perform a wide range of operations such as creating database structures, inserting data, updating existing records, deleting unwanted data, and retrieving informations using queries. SQL is extremely important in DBMS because almost every modern application - whether it is a banking system, e-commerce website, hospital management system, or social media platform - stores data in relational databases and relies on SQL to access that data. It provides a simple, English like syntax that makes database interaction easy even for non programmers. SQL also supports transaction management, security and data integrity which are essential for any production level system.

ques 2. what is the difference b/w DDL and DML

Ans DDL stands for Data Definition Language. It is used to define, create and modify the structure of database objects such as tables, schemas

INDEX, and views. DDL commands include CREATE (to create a table or database), ALTER (to modify an existing structure) DROP (to delete a table) and TRUNCATE (to remove all rows from a table). DDL commands are auto committed - meaning changes cannot be rolled back.

DML stands for Data Manipulation Language. It's used to manipulate or work with the actual data stored inside the tables. DML commands include INSERT (to add new rows)

UPDATE (to modify existing data), DELETE (to remove specific rows) and SELECT (to retrieve data).

Unlike DDL, DML commands are not auto committed so they can be rolled back if needed - making them safe for transactional operations.

Q3. What is a JOIN? Explain its types with examples.

Ans. A JOIN in SQL is used to combine rows from two or more tables based on a related column b/w them. It's one of the most powerful features of SQL as it allows us to retrieve meaningful combined data from multiple tables at once.

Types of JOIN

a) INNER JOIN: Returns only the rows that have matching value in both tables. Rows with no match in either table are excluded from the result.

Joining student and marks tables to get students who have marks.

b) LEFT JOIN: Return all rows from the left table and the matching rows from the right table. If there is no match, NULL is shown.
Ex: All students and their marks

c) RIGHT JOIN: Return all rows from the right table and matching rows from the left table. NULL appear for unmatched left table.

d) FULL OUTER JOIN: Return all rows from both tables. Where there is no match, NULL are filled in for the missing side.
Combines LEFT and RIGHT JOIN.

Q4. What is Group BY and Having clause?
How are they used?

The Group BY clause in SQL is used to group rows that share the same value in one or more columns into summary or aggregate rows. It's typically used with aggregate functions like COUNT(), SUM(), AVG(), MAX() to produce a single result for each group.
For ex: if we want to count how many students belong to each department, we use GROUP BY the department column.

The HAVING clause is used to filter the results after the grouping has been done. It works like the WHERE clause but specifically for grouped data. WHERE cannot be used with aggregate functions, so HAVING was introduced for this purpose.

Ex:

```
SELECT department, COUNT(*) AS total  
FROM student  
GROUP BY department  
HAVING COUNT(*) > 10;
```

This gives only those departments which have more than 10 students each.

Ques 5 What is subquery? Explain with a real example.
A subquery (also called an inner query or nested query) is a SQL query that is written inside another query, known as the outer query. The result of subquery is used as a condition or input for the outer query. Subqueries make complex queries more readable and break down the problem into smaller manageable parts.

Subqueries can appear in WHERE clause, FROM clause, or SELECT clause.

Real life Example: In a university database, to find all the student who scored above the class average in their examination

```
SELECT name, marks.  
FROM student  
WHERE mark > (SELECT AVG(marks) FROM student);
```

Here, the inner query calculate the average marks of all student, and the outer query retrieve only those student who scored higher than that average. Subqueries can also be correlated, where the inner query reference a column from the outer query and execute once for each row - making them very powerful for advanced filtering.

Ques 6 What is Relational Algebra? Why is it important?
Relational Algebra is a procedural query language that form the theoretical and mathematical foundation of SQL and relational databases. It work on relation (tables) & produces a new relation as output, making it a closed algebra. It was proposed by E.F. Codd in 1970 as part of his relational model.

Fundamental operations in Relational Algebra.

- Select (σ / Π): Filter row based on a condition, like WHERE in SQL.

- Project (π): Select specific columns from a relation like SELECT
 - Union ($\cup$): Combine tuple from two compatible relations.
 - Difference ($-$): Return tuple in first relation but not in second.
 - Cartesian Product ($\times$): Combine each row of one relation with every row of another.
 - Join ($\Join$): Combine relations based on a matching condition.
- Relational Algebra is imp bcz it gives DBMS developers a formal way to evaluate & optimize queries. Every SQL queries is internally converted into relational Algebra expression by the query optimizer before execution.

Ques 10 What is difference between WHERE and HAVING clause?

WHERE and HAVING are both used to filter data in SQL, but they operate at different stage of query execution & serve different purpose.

WHERE clause:

- Filter individual rows BEFORE any grouping is done by GROUP BY.
- It cannot be used with aggregate function like COUNT(), SUM(), AVG().
- Applies directly on raw table data during the selecting phase.
- Example: `SELECT * FROM student WHERE marks > 50.`

HAVING clause:

- Filter grouped row AFTER the group BY operation has been completed.
- It can be used with aggregate functions.
- Used specifically to place condition on aggregated results.
- Example: `SELECT dept, AVG(marks) FROM student GROUP BY dept HAVING AVG(marks) > 60;`

In simple terms: WHERE works on rows before grouping, HAVING works on groups after aggregation. Both can be used together in the same query.

Section - 'B'

- Q1. Write an SQL query to retrieve all students with marks > 75.
To retrieve all students who have scored more than 75.

```
SELECT student_id, name, department, marks  
FROM student  
WHERE marks > 75  
ORDER BY marks DESC;
```

- Q2. Write a query using JOIN to combine Student and Course.

```
SELECT s.student_id, s.name, s.department, c.course_id,  
       c.course_name, c.instructor.
```

```
FROM student s
```

```
INNER JOIN course c
```

```
ON s.course_id = c.course_id
```

```
ORDER BY s.name;
```

- Q3. Write a query using GROUP BY to count students per department.

```
SELECT department,
```

```
       COUNT(*) AS total_students,
```

```
       AVG(marks) AS average_marks
```

```
FROM student
```

```
GROUP BY department
```

```
ORDER BY total_students DESC
```


Section - C

Q1 A Company database stores employee and department data. Write a query to find employees working in a specific department.

In a Company database, the Employee table and Department table are typically linked through a dept-id foreign key. To find all employees in a specific department (eg. "Sales"), we JOIN both tables and filter using the WHERE clause.

```
SELECT e.emp-id, e.emp-name, e.designation,  
       e.salary, d.dept-name  
FROM employee e  
INNER JOIN department d  
      ON e.dept-id = d.dept-id  
WHERE d.dept-name = 'Sales'  
ORDER BY e.emp-name;
```

This query uses INNER JOIN to link employees with their respective department and then filters to show only those belonging to the Sales department. Sorting alphabetically by name makes the result easy to read for HR or management reports.

Ques 2 An e-commerce system stores orders. Retrieve total sales per customer.

In an e-commerce platform, the orders table holds individual orders records for every customer. To find the total amount spent by each customer, we use GROUP BY on the customer-id column along with the SUM() aggregate function.

```
SELECT c.customer-id, c.customer-name,  
       COUNT(o.order-id) AS total-orders,  
       SUM(o.order-amount) AS total-sales,  
       MAX(o.order-date) AS last-date  
FROM customer c
```

Section - C

Q1 A Company database stores employee and department data. Write a query to find employees working in a specific department.

In a company database, the Employee table and Department table are typically linked through a dept-id foreign key. To find all employees in a specific department (eg. "Sales"), we JOIN both tables and filter using the WHERE clause.

```
SELECT e.emp-id, e.emp-name, e.designation,  
       e.salary, d.dept-name  
FROM employee e  
INNER JOIN department d  
      ON e.dept-id = d.dept-id  
WHERE d.dept-name = 'Sales'  
ORDER BY e.emp-name;
```

This query uses INNER JOIN to link employees with their respective department and then filters to show only those belonging to the Sales department. Sorting alphabetically by name makes the result easy to read for HR or management reports.

Ques 2 An e-commerce system stores orders. Retrieve total sales per customer.

In an e-commerce platform, the orders table holds individual orders records for every customer. To find the total amount spent by each customer, we use GROUP BY on the customer-id column along with the SUM() aggregate function.

```
SELECT c.customer-id, c.customer-name,  
       COUNT(o.order-id) AS total-orders,  
       SUM(o.order-amount) AS total-sales,  
       MAX(o.order-date) AS last-date  
FROM customer c
```


LEFT JOIN Orders O

ON C.customer_id = O.customer_id

GROUP BY C.customer_id, C.customer_name

ORDER BY total_sales DESC;

A left JOIN is used so that even customers with no order appears.

Ques 3. A University database needs top 3 student. Write query.
To find the top 3 students based on their marks from a university database, we sort the student records in descending order of marks and then use LIMIT to restrict the output to only the top 3 results.

```
SELECT student_id, name, department,  
marks, grade
```

```
FROM student
```

```
WHERE marks IS NOT NULL
```

```
ORDER BY marks DESC
```

```
LIMIT 3;
```

ORDER BY marks DESC sorts students from highest to lowest marks. LIMIT 3 restricts the result to only three rows - the top 3 performers. The WHERE clause ensures that students with NULL (missing) marks are excluded from the ranking. This query is useful for generating merit lists, awarding scholarships, or prize distribution.

Sec - 'D'

Task : Write an SQL query for a real-life system and explain its logic.

System : Hospital Management System

Task : Find all appointments scheduled for a specific doctor on a given date, along with patient details and the reason for the visit.

```
SELECT p.patient_id, p.patient_name, p.age,  
       a.appointment_date, a.time_slot,  
       a.reason, d.doctor_name, d.specialization.  
  
FROM appointment a  
INNER JOIN patient p ON a.patient_id = p.patient_id  
INNER JOIN doctor d ON a.doctor_id = d.doctor_id  
WHERE d.doctor_name = 'DR. RAHUL'  
AND a.appointment_date = "2026-13-15"  
ORDER BY a.time_slot;
```

- Logic & Insight : This query uses two INNER JOINS to connect three related tables.
- It retrieves only relevant appointments by filtering on both the doctor name and
- the appointment date simultaneously using the AND operator in the WHERE clause.
- Ordering by time_slot ensures the receptionist sees appointments in chronological.
- Order - making this query directly usable in a real hospital scheduling system.

Task 2 : Write a JOIN query and mention one use case :

System : Online learning Platform (like Coursera)

Task : Find all students enrolled in a particular

course along with their progress.

```
SELECT s.student_id, s.name, s.email, c.course_name, c.instructor_id,  
       e.enrollment_date, e.progress, p.percentage  
FROM student s
```

```
INNER JOIN enrollment e ON s.student_id = e.student_id
```

```
INNER JOIN course c ON e.course_id = c.course_id
```

```
WHERE c.course_name = 'Database Management Systems'
```

```
AND e.progress_percentage < 100
```

```
ORDER BY e.progress_percentage DESC;
```

→ Use case: This query is used by instructors to monitor which students are still

→ in progress for their course. Filtering progress < 100 excludes already-completed

→ students. The result helps the platform send reminder notifications or offer extra

→ help sessions to students who are falling behind on their course completion.

Task 3: Write a GROUP BY query and explain its output in detail.

System: School Academic Result Analysis

Task: Find the average, highest and lowest marks for each subject across all students and identify which subjects students are struggling with the most.

```
SELECT subject_name,  
       COUNT(student_id) AS students_appeared,  
       ROUND(AVG(marks), 2) AS average_marks,  
       MAX(marks) AS highest_marks,  
       MIN(marks) AS lowest_marks,  
       SUM(CASE WHEN marks < 35 THEN 1 ELSE  
              0 END) AS failed_count
```

- FROM result
- GROUP BY subject-name
- ORDER BY average-marks ASC;
- ⇒ Output Explanation: Each row of output represents one subject. The query shows
 - how many students appeared, what the average marks were, the highest and lowest
 - scores and a count of how many students failed (below 35 marks). Ordering by
 - average-marks ASC places the most difficult subjects (lowest averages) at the
 - top. This type of analysis is very useful for school management to take targeted
 - academic interventions for underperforming subjects.